Экспортируй анимацию в MP4 формат

Ниже — обновлённый скрипт `model_animation_mp4.py`, который экспортирует анимацию в MP4. Он проверяет наличие Python-библиотек и наличие `ffmpeg`; без `ffmpeg` MP4 не сохранится, поэтому скрипт предложит установить его или сохранить GIF как резервный вариант. Matplotlib использует `FFMpegWriter` для записи MP4, а кодек по умолчанию — `h264`.[1] [2] [3]

Сохрани код как `model_animation_mp4.py` на рабочий стол и запусти:

```
python model_animation_mp4.py
```

```
# -*- coding: utf-8 -*-
"""
Анимация универсальной модели ответственности:

V(T) = B(T) - R(T) + ∫ D(t) * S(t) dt

D(t) = exp(-lambda * t)

Экспорт в MP4 через FFMpegWriter.
Перед запуском проверяются:
- numpy
- matplotlib
- pillow
- ffmpeg
"""

import importlib.util
import shutil
import subprocess
import sys

PYTHON_PACKAGES = ["numpy", "matplotlib", "pillow"]


def check_python_packages():
    missing = []

    for package in PYTHON_PACKAGES:
        spec = importlib.util.find_spec(package)
        if spec is None:
            missing.append(package)
            print(f"[НЕТ] {package} не установлен.")
        else:
```

```
            print(f"[OK] {package} установлен.")

    if missing:
        answer = input(
            f"\nДля работы нужны: {', '.join(missing)}.\n"
            "Установить их сейчас через pip? [y/n]: "
        ).strip().lower()

        if answer in ("y", "yes", "д", "да"):
            for package in missing:
                print(f"Устанавливаю {package}...")
                subprocess.check_call(
                    [sys.executable, "-m", "pip", "install", package]
                )
        else:
            print("\nУстановка отменена. Скрипт завершён.")
            sys.exit(1)

    for package in PYTHON_PACKAGES:
        if importlib.util.find_spec(package) is None:
            print(f"Ошибка: {package} не удалось установить.")
            sys.exit(1)

    print("\nPython-библиотеки готовы.\n")


def check_ffmpeg():
    ffmpeg_path = shutil.which("ffmpeg")

    if ffmpeg_path is None:
        print("\n[НЕТ] ffmpeg не найден в системе.")
        print("MP4 требует установленный ffmpeg.")
        print("Установи ffmpeg и добавь его в PATH.")
        print("Примеры:")
        print("Windows: скачай с https://ffmpeg.org и добавь в PATH")
        print("macOS: brew install ffmpeg")
        print("Linux: sudo apt install ffmpeg")

        answer = input(
            "\nПродолжить и сохранить анимацию как GIF вместо MP4? [y/n]: "
        ).strip().lower()

        if answer in ("y", "yes", "д", "да"):
            return False
        else:
            sys.exit(1)

    print(f"[OK] ffmpeg найден: {ffmpeg_path}")
    return True


check_python_packages()
ffmpeg_available = check_ffmpeg()

import numpy as np
import matplotlib.pyplot as plt
```

```
from matplotlib.animation import FuncAnimation, FFMpegWriter, PillowWriter


# ----------------------------
# Модель
# ----------------------------

def delay_coefficient(t, lam=0.15, critical_time=1.0):
    t = np.asarray(t, dtype=float)
    return np.where(
        t <= critical_time,
        1.0,
        np.exp(-lam * (t - critical_time))
    )


def model_value(B, R, S, lam, critical_time=1.0, T=20.0, steps=1000):
    t = np.linspace(0, T, steps)
    dt = t[^1] - t[^0]
    D = delay_coefficient(t, lam, critical_time)
    support_integral = np.sum(D * S * dt)
    return B - R + support_integral


# ----------------------------
# Параметры анимации
# ----------------------------

T_MAX = 20.0
FRAMES = 120
LAMBDA = 0.18
CRITICAL_TIME = 1.0

t_values = np.linspace(0, T_MAX, 500)

B = 100.0
R = 80.0
S = 60.0

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

# Верхний график: коэффициент задержки
line_D, = ax1.plot(
    [], [],
    color="#c0392b",
    linewidth=3,
    label="Эффективность помощи D(t)"
)

ax1.set_xlim(0, T_MAX)
ax1.set_ylim(0, 1.1)
ax1.set_xlabel("Время задержки t")
ax1.set_ylabel("D(t)")
ax1.set_title("Задержка помощи: чем позже реакция, тем ниже эффективность")
ax1.grid(True, alpha=0.3)
ax1.legend(loc="upper right")
```

```
# Нижний график: V(T)
line_V, = ax2.plot(
    [], [],
    color="#2e86de",
    linewidth=3,
    label="Итоговая ценность V(T)"
)

ax2.set_xlim(0, T_MAX)
ax2.set_ylim(-20, 100)
ax2.set_xlabel("Время задержки t")
ax2.set_ylabel("V(T)")
ax2.set_title("Итоговая ценность системы V(T)")
ax2.grid(True, alpha=0.3)
ax2.legend(loc="upper right")

# Текст с текущими значениями
text = ax2.text(
    0.02, 0.90,
    "",
    transform=ax2.transAxes,
    fontsize=11,
    family="monospace",
    bbox=dict(facecolor="white", alpha=0.8)
)


def update(frame):
    current_t = T_MAX * frame / (FRAMES - 1)

    D_current = delay_coefficient(
        np.array([current_t]),
        lam=LAMBDA,
        critical_time=CRITICAL_TIME
    )[^0]

    V_current = model_value(
        B=B,
        R=R,
        S=S,
        lam=LAMBDA,
        critical_time=CRITICAL_TIME,
        T=current_t
    )

    t_plot = t_values[t_values <= current_t]
    D_plot = delay_coefficient(
        t_plot,
        lam=LAMBDA,
        critical_time=CRITICAL_TIME
    )

    V_plot = []
    for time_point in t_plot:
        value = model_value(
```

```
            B=B,
            R=R,
            S=S,
            lam=LAMBDA,
            critical_time=CRITICAL_TIME,
            T=time_point
        )
        V_plot.append(value)

    line_D.set_data(t_plot, D_plot)
    line_V.set_data(t_plot, V_plot)

    text.set_text(
        f"t = {current_t:.2f}\n"
        f"D(t) = {D_current:.3f}\n"
        f"V(T) = {V_current:.2f}"
    )

    return line_D, line_V, text


ani = FuncAnimation(
    fig,
    update,
    frames=FRAMES,
    interval=60,
    blit=False,
    repeat=True
)

if ffmpeg_available:
    output_file = "model_delay_animation.mp4"

    writer = FFMpegWriter(
        fps=30,
        bitrate=2400,
        metadata={
            "title": "Universal Responsibility Model",
            "artist": "Perplexity Model"
        }
    )

    ani.save(
        output_file,
        writer=writer,
        dpi=150
    )

    print(f"Готово. Создан файл: {output_file}")

else:
    output_file = "model_delay_animation.gif"

    writer = PillowWriter(fps=15)
    ani.save(
        output_file,
```

```
        writer=writer,
        dpi=120
    )

    print(f"ffmpeg не найден. Создан резервный файл: {output_file}")
```

Что изменилось

- Добавлена проверка `ffmpeg` через `shutil.which("ffmpeg")`.
- Если `ffmpeg` есть — анимация сохраняется в `model_delay_animation.mp4`.
- Если `ffmpeg` нет — скрипт предлагает сохранить GIF как резерв.
- MP4 пишется через `FFMpegWriter`, который передаёт кадры напрямую в `ffmpeg`. [2] [3] [1]

Как установить ffmpeg

- **Windows:** скачай сборку с официального сайта `ffmpeg.org`, распакуй и добавь папку `bin` в PATH.
- **macOS:**
  ```
  brew install ffmpeg
  ```
- **Linux/Ubuntu:**
  ```
  sudo apt install ffmpeg
  ```

Проверка установки

После установки проверь в терминале:

```
ffmpeg -version
```

Если команда выводит версию — MP4 будет сохраняться нормально.

Финальный файл

После успешного запуска появится:

```
model_delay_animation.mp4
```

Если `ffmpeg` не установлен, появится:

```
model_delay_animation.gif
```

Главная формула

$$\boxed{V(T) = B(T) - R(T) + \int_0^T e^{-\lambda t} S(t) dt}$$

Анимация в MP4 показывает:

- как падает $D(t)$;
- как падает $V(T)$;
- как промедление превращает помощь в частичную компенсацию;
- как задержка увеличивает итоговый ущерб.

Финальный смысл

MP4-анимация показывает не просто формулу, а процесс утраты ценности помо

Именно это делает модель наглядной: видно, что **поздняя помощь уже не равна ранней помощи.** [4] [1] [2]

⁂

1. https://matplotlib.org/3.8.0/api/animation_api.html
2. https://matplotlib.org/3.7.4/api/_as_gen/matplotlib.animation.FFMpegWriter.html
3. https://github.com/matplotlib/matplotlib/blob/v3.9.1/lib/matplotlib/animation.py
4. https://matplotlib.org/3.8.4/gallery/animation/dynamic_image.html
5. https://discourse.matplotlib.org/t/write-each-frame-to-picture-with-ffmpegwriter/24663
6. https://matplotlib.org/stable/users/explain/animations/animations.html
7. https://matplotlib.org/stable/api/animation_api.html
8. https://matplotlib.org/3.7.5/tutorials/introductory/animation_tutorial.html
9. https://matplotlib.org/3.9.0/users/explain/animations/animations.html
10. https://matplotlib.org/stable/api/_as_gen/matplotlib.animation.FFMpegWriter.html
11. https://fr.matplotlib.net/stable/api/_as_gen/matplotlib.animation.FFMpegWriter.html
12. https://www.resus.org.uk/professional-library/2025-resuscitation-guidelines/ethics-guidelines
13. https://matplotlib.org/3.8.0/_downloads/b5546881188fb3e3d279d5a734f9240d/animations.py
14. https://datafield.dev/data-visualization-python/part-03/chapter-15/
15. https://en.matplotlib.net/stable/_sources/api/_as_gen/matplotlib.animation.FFMpegWriter.rst.txt
16. https://github.com/catkira/BlochBuster/blob/master/FFMPEGwriter.py